Contenido

Comienzo el frontend instalando las librerías necesarias 2

Con toda la configuración lista creo el punto de entrada de la aplicación creando el componente padre que contendrá la interfaz 5

Preparación de la vista Blade 6

Creo el componente App.vue para comprobar que todo funciona 6

Arranco los dos servidores en paralelo (Laravel y Vue) para probar la app 7

Me encuentro con un problema y es que veo que si me muestra el título, por lo que se está cargando el welcome.blade, pero no se debe estar cargando el componente ya que veo una pantalla en blanco..... 7

Hago la configuración necesaria para Codespaces 8

Vuelvo a correr los dos servidores en paralelo..... 9

El siguiente reto técnico es que, al tener uan base de datos con 50.000 registros, en el frontend no puedo mostrar los 50.000 registros, por lo que debo usar la técnica de paginación 10

Pongo a prueba los cambios creando un test automatizado..... 11

Con los test superados ya puedo ir creando los componentes 13

El componente de filtrado lo hago en un componente a parte para separar responsabilidades. 15

Cierro la feature..... 17

Corro los servidores y abro la app en el navegador 17

Abro la app en el navegador:..... 18

Pruebo la funcionalidad de los filtros 18

Aplico el filtro y compruebo que tengo un bug. Me devuelve todos los tickets de nuevo sin filtrar 19

Combino todos los filtros para probarlos 22

Hago el commit y actualizo el repositorio 23

Comienzo a implementar la funcionalidad de la exportación masiva. 23

Hago la prueba final del funcionamiento de la exportación primero con filtro:. 25

Hago la exportación de los 50.000 tickets (debo optimizar) 27

Próximos pasos en el proyecto general (usuarios y autenticación, roles, edición de tickets y pentesting)..... 28

Cierre de la PoC 29

Guardo el repositorio en mi equipo personal:..... 29
Repositorio público:..... 30

Comienzo el frontend instalando las librerías necesarias

La arquitectura que he pensado para el frontend es una SPA (Single Page Application) usando Vue 3, Tailwind CSS y Vite.

Instalo las dependencias base que trae Laravel por defecto:

npm install

```
● @ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $ npm install

  added 82 packages, and audited 83 packages in 16s

  21 packages are looking for funding
    run `npm fund` for details

  found 0 vulnerabilities
  npm notice
  npm notice New minor version of npm available! 11.6.2 -> 11.10.1
  npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.10.1
  npm notice To update run: npm install -g npm@11.10.1
  npm notice
```

Instalo el núcleo de Vue, el enrutador y Axios (para conectar con mi API):

npm install vue vue-router axios @vitejs/plugin-vue

```
@ValentinuxX ➜ /workspaces/PoC-Exportation-Project (develop) $ npm install vue vue-router axios @vitejs/plugin-vue
added 61 packages, and audited 144 packages in 8s

37 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

Instalo Tailwind CSS para el diseño de la interfaz:

npm install -D tailwindcss@3 postcss autoprefixer

```
● @ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $ npm install -D tailwindcss@3 postcss autoprefixer

added 61 packages, changed 1 package, and audited 214 packages in 5s

54 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

Genero los archivos de configuración del diseño:

npx tailwindcss init -p

```
● @ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $ npx tailwindcss init -p

Created Tailwind CSS config file: tailwind.config.js
Created PostCSS config file: postcss.config.js
```

Edito el archivo de configuración de Tailwind:

```
JS tailwind.config.js U ×
JS tailwind.config.js > ...
1   /** @type {import('tailwindcss').Config} */
2   export default {
3     content: [
4       "./resources/**/*.blade.php",
5       "./resources/**/*.js",
6       "./resources/**/*.vue",
7     ],
8     theme: {
9       extend: {},
10    },
11    plugins: [],
12  }
13
14
```

Inyecto Tailwind en el CSS editando el archivo app.css:

```
JS tailwind.config.js U    # app.css M ×
resources > css > # app.css
1  @tailwind base;
2  @tailwind components;
3  @tailwind utilities;
```

Conecto Vue con el motor de Vite editando el archivo vite.config.js:

```
vite.config.js M ×
vite.config.js > ...
1  import { defineConfig } from 'vite';
2  import laravel from 'laravel-vite-plugin';
3  import vue from '@vitejs/plugin-vue'; // 1. Importo el plugin de Vue
4
5  export default defineConfig({
6      plugins: [
7          laravel({
8              input: ['resources/css/app.css', 'resources/js/app.js'],
9              refresh: true,
10         }),
11         vue({ // 2. Le decimos a Vite que use el plugin
12             template: {
13                 transformAssetUrls: {
14                     base: null,
15                     includeAbsolute: false,
16                 },
17             },
18         }),
19     ],
20 });
21
22
```

Con toda la configuración lista creo el punto de entrada de la aplicación creando el componente padre que contendrá la interfaz

Creo el archivo App.vue en /resources/js/ creando un nuevo componente de Vue que será el punto de entrada de la aplicación:

```
JS app.js
resources > js > JS app.js
1  import './bootstrap';
2  import { createApp } from 'vue';
3  import App from './App.vue';
4
5  //Creación la app de Vue e inyección en el div con id="app"
6  createApp(App).mount('#app');
7
```

Preparación de la vista Blade

Limpio la vista por defecto de Laravel (resources/views/welcome.blade.php) para dejarla como un contenedor vacío. Inyecto los assets compilados mediante Vite y creo el nodo raíz del DOM para Vue.

```
welcome.blade.php
resources > views > welcome.blade.php
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="utf-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1">
6      <title>PoC Tickets</title>
7
8      @vite(['resources/css/app.css', 'resources/js/app.js'])
9  </head>
10 <body class="antialiased">
11     <div id="app"></div>
12 </body>
13 </html>
14
15
```

Creo el componente App.vue para comprobar que todo funciona

En la carpeta resources/js/ creo el componente App.vue con el contenido para la primera comprobación para ver si el puente entre Laravel y Vue:

```
App.vue M ×
resources > js > App.vue > { } script setup
1  <template>
2    <div class="min-h-screen bg-gray-100 flex items-center justify-center">
3      <div class="bg-white p-8 rounded-xl shadow-lg text-center">
4        <h1 class="text-3xl font-bold text-blue-600 mb-4">¡Vue y Tailwind están unidos</h1>
5        <p class="text-gray-600">viendo esto la arquitectura SPA funciona.</p>
6      </div>
7    </div>
8  </template>
9
10 <script setup>
11 // Aquí irá la lógica de Javascript más adelante
12 </script>
```

Arranco los dos servidores en paralelo (Laravel y Vue) para probar la app

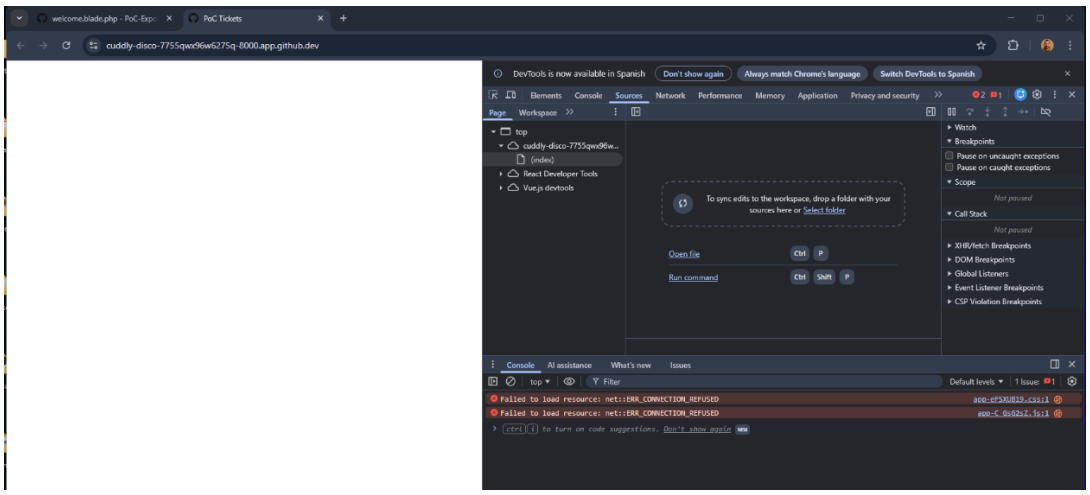
```
@ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $ npm run dev
  VITE v7.3.1  ready in 3144 ms

  ➜ Local:   http://localhost:5173/
  ➜ Network: use --host to expose
  ➜ press h + enter to show help

  LARAVEL v12.51.0  plugin v2.1.0

  ➜ APP_URL: http://localhost
```

```
@ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $
@ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $ php artisan serve
  INFO  Server running on [http://127.0.0.1:8000].

  Press Ctrl+C to stop the server
```

Me encuentro con un problema y es que veo que si me muestra el título, por lo que se está cargando el welcome.blade, pero no se debe estar cargando el componente ya que veo una pantalla en blanco

Después de investigar entiendo que el problema es que cuando ejecuto el npm run dev Vite levanta un servidor en http://localhost:5173, pero en Codespaces mi navegadorintenta buscar el localhost en mi propio ordenador físico y al no encontrar nada el Javascript de Vue nunca llega a ejecutarse y la pantalla se queda en blanco.

Eso lo puedo corroborar al ver que en la consola del navegador encuentro el error Fallo al cargar recurso (ERR_CONNECTION_REFUSED)



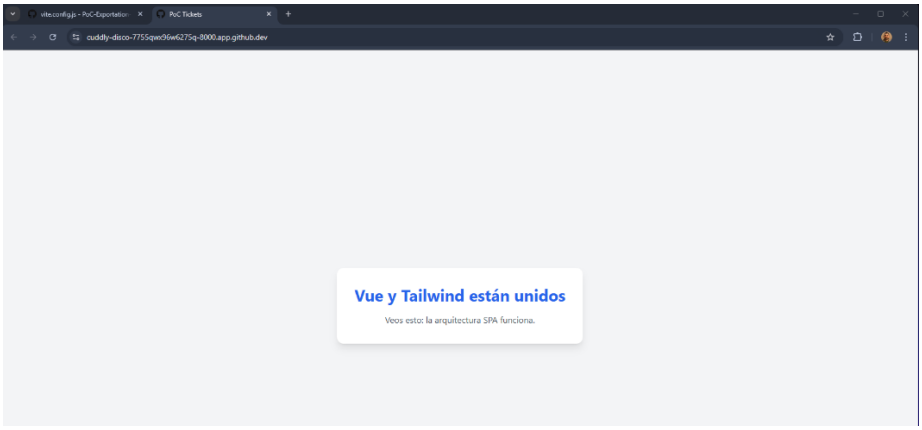
Hago la configuración necesaria para Codespaces

Sustituyo el contenido de vite.config.js con el bloque server que le indica como comportarse en Codespaces:

```jsx
vite.config.js
import { defineConfig } from 'vite';
import laravel from 'laravel-vite-plugin';
import vue from '@vitejs/plugin-vue'; // 1. Importo el plugin de Vue

export default defineConfig({
    // configuración del servidor indicando que el host es el de Codespaces
  server: {
        host: '0.0.0.0',
        port: 5173,
        strictPort: true, // Trato de evitar que cambie de puerto sin avisar
        cors: true,
        headers: {
            'Access-Control-Allow-Origin': '*' // Inyecto la cabecera a la fuerza
        },
        hmr: {
            host: 'cuddly-disco-7755qwx96w6275q-5173.app.github.dev',
            protocol: 'wss',
            clientPort: 443
        },
    },
    plugins: [
        laravel({
            input: ['resources/css/app.css', 'resources/js/app.js'],
            refresh: true,
        }),
        vue({ // 2. Le decimos a Vite que use el plugin
            template: {
                transformAssetUrls: {
                    base: null,
                    includeAbsolute: false,
                },
            },
        }),
    ],
});
```

Vuelvo a correr los dos servidores en paralelo

Al correr los dos servidores en paralelo ya puedo ver que el puente se ha creado y mi componente se muestra en el navegador:



El siguiente reto técnico es que, al tener uan base de datos con 50.000 registros, en el frontend no puedo mostrar los 50.000 registros, por lo que debo usar la técnica de paginación

Para usar la técnica de paginación debo implementarla en el controlador y en el servicio.

Primero refactorizo el servicio modificando el getAllTickets para que acepte un array de filtros y devuelva la paginación, solo que para no repetir todo el bloque if en ambos métodos lo que hago es aislar los filtros en una función privada que construya la consulta con los filtros. Así, tanto la web (Vue) como la exportación usarán el mismo motor de búsqueda.

Modifico el TicketService

PoC-Exportation-Project [Codespaces: cuddly disco]

TicketService.php M

app > Services > TicketService.php > TicketService > buildFilterQuery

```php
<?php

namespace App\Services;

use App\Models\Ticket;

class TicketService
{
    /**
     * MÉTODO PRiVADO: Construye la consulta base con los filtros.
     * Así no repito este código gigante en la paginación y en la exportación.
     */
    private function buildFilterQuery(array $filters = [])
    {
        $query = Ticket::with(['user', 'category']);

        if (!empty($filters['status'])) {
            $query->where('status', $filters['status']);
        }

        if (!empty($filters['priority'])) {
            $query->whereHas('category', function ($q) use ($filters) {
                $q->where('priority', $filters['priority']);
            });
        }

        if (!empty($filters['category_id'])) {
            $query->where('category_id', $filters['category_id']);
        }

        if (!empty($filters['user_id'])) {
            $query->where('user_id', $filters['user_id']);
        }

        if (!empty($filters['date_from'])) {
            $query->whereDate('created_at', '>=', $filters['date_from']);
        }
```

```php
        if (!empty($filters['date_to'])) {
            $query->whereDate('created_at', '<=', $filters['date_to']);
        }

        if (!empty($filters['search'])) {
            $query->where('title', 'like', '%' . $filters['search'] . '%');
        }

        return $query;
    }

    /**
     * Obtiene los tickets filtrados y PAGINADOS (Para la tabla de Vue.js)
     */
    public function getAllTickets(array $filters = [])
    {
        // Usamos la consulta base y le aplicamos paginate(15)
        return $this->buildFilterQuery($filters)
                    ->orderBy('created_at', 'desc')
                    ->paginate(15);
    }

    /**
     * Crear un nuevo ticket en la base de datos.
     */
    public function createTicket(array $data)
    {
        return Ticket::create($data);
    }
```

```php

    /**
     * Obtener un ticket específico por su ID.
     */
    public function getTicketById(string $id)
    {
        return Ticket::with(['user', 'category'])->findOrFail($id);
    }

    /**
     * Actualizar un ticket existente.
     */
    public function updateTicket(string $id, array $data)
    {
        $ticket = $this->getTicketById($id);
        $ticket->update($data);
        return $ticket;
    }

    /**
     * Eliminar un ticket de la base de datos.
     */
    public function deleteTicket(string $id)
    {
        $ticket = $this->getTicketById($id);
        $ticket->delete();
        return true;
    }

    /**
     * Obtener los tickets filtrados para exportación.
     * NOTA: Solo devolvemos la query para que el ExportService pueda usar ->cursor() y hacer streaming.
     */
    public function getTicketsForExport(array $filters = [])
    {
        return $this->buildFilterQuery($filters)->orderBy('created_at', 'desc');
    }
}
```

Pongo a prueba los cambios creando un test automatizado

Para poner a prueba los cambios no puedo usar RefreshDatabase ya que borraría toda la base de datos así que vuelvo a usar la técnica de la base de datos SQLite en memoria RAM.

Compruebo que tengo configurado mi phpunit.xml con las líneas necesarias:

```
26        <env name="DB_CONNECTION" value="sqlite"/>
27        <env name="DB_DATABASE" value=":memory:"/>
```

Creo el archivo de test:

php artisan make:test TicketServiceTest

Y programo el test que acabo de crear:

```
TicketServiceTest.php ×
tests > Feature > TicketServiceTest.php > TicketServiceTest
1   <?php
2
3   namespace Tests\Feature;
4
5   use Illuminate\Foundation\Testing\RefreshDatabase;
6   use Tests\TestCase;
7   use App\Models\Ticket;
8   use App\Models\Category;
9   use App\Models\User;
10  use App\Services\TicketService;
11
12  class TicketServiceTest extends TestCase
13  {
14      // Esto asegura que la base de datos temporal se limpie entre cada test
15      use RefreshDatabase;
16
17      protected TicketService $ticketService;
18
19      protected function setUp(): void
20      {
21          parent::setUp();
22          // Instanciamos el servicio que vamos a poner a prueba
23          $this->ticketService = new TicketService();
24      }
25
26      /**
27       * Prueba 1: Verifica que si hay 20 tickets, el servicio devuelve solo 15 en la primera página.
28       */
29      public function test_get_all_tickets_returns_paginated_results()
30      {
31          // 1. Arrange (Preparar el entorno): Creamos 20 tickets falsos en la BD en memoria
32          Ticket::factory()->count(20)->create();
33
34          // 2. Act (Actuar): Llamamos a nuestro servicio sin filtros
35          $result = $this->ticketService->getAllTickets();
36
37          // 3. Assert (Comprobar): Verificamos que devuelve 15 elementos y que el total es 20
38          $this->assertCount(15, $result->items());
39          $this->assertEquals(20, $result->total());
40      }
```

```php
    /**
     * Prueba 2: Verifica que el motor de búsqueda y filtros funciona correctamente.
     */
    public function test_get_all_tickets_applies_filters_correctly()
    {
        // 1. Arrange: Creamos categorías y tickets muy específicos
        $categoriaFallo = Category::factory()->create(['name' => 'Fallo Crítico']);
        $categoriaDuda = Category::factory()->create(['name' => 'Duda General']);

        // Creamos 5 tickets de "Fallo Crítico" en estado "Abierto"
        Ticket::factory()->count(5)->create([
            'category_id' => $categoriaFallo->id,
            'status' => 'Abierto',
            'title' => 'Tubería rota'
        ]);

        // Creamos 3 tickets de "Duda General" en estado "Cerrado"
        Ticket::factory()->count(3)->create([
            'category_id' => $categoriaDuda->id,
            'status' => 'Cerrado',
            'title' => 'Horario ayuntamiento'
        ]);

        // 2. Act: Le pedimos al servicio que filtre solo los Fallos Críticos Abiertos
        $filters = [
            'category_id' => $categoriaFallo->id,
            'status' => 'Abierto'
        ];

        $result = $this->ticketService->getAllTickets($filters);

        // 3. Assert: Debería devolvernos exactamente los 5 tickets que coinciden
        $this->assertCount(5, $result->items());
        $this->assertEquals('Abierto', $result->first()->status);
        $this->assertEquals($categoriaFallo->id, $result->first()->category_id);
    }
}
```

Nota: en este test ya comienzo a poner los nombres de los métodos en inglés, ya que es el modo profesional de hacerlo.

php artisan test --filter TicketServiceTest

```
● @ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $ php artisan test --filter TicketServiceTest

  PASS  Tests\Feature\TicketServiceTest
  ✓ get all tickets returns paginated results
  ✓ get all tickets applies filters correctly

  Tests:    2 passed (5 assertions)
  Duration: 1.42s

○ @ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $
```

Pasa los dos test.

Con los test superados ya puedo ir creando los componentes

Empiezo creando el componente principal modificando el contenido de prueba del App.vue e implementando tickets y loading como ref (para que sean reactivas) y si su valor cambia el HTML se actualice.

También uso onMounted para decirle a Vue que, en cuanto termine de pintar el HTML vacío en la pantalla del usuario, ejecute la función fetchTickets() para ir a buscar los datos a mi API.

Como la API de Laravel devuelve un objeto paginado, los tickets reales vienen dentro de la propiedad .data. Con v-for="ticket in tickets.data Vue los recorre y crea una fila <tr> por cada uno.

```
App.vue M ×
resources > js > App.vue > { } script setup
1   <template>
2     <div class="min-h-screen bg-gray-100 p-8">
3       <div class="max-w-7xl mx-auto">
4
5         <div class="mb-6 flex flex-col md:flex-row justify-between items-start md:items-end gap-4">
6           <div>
7             <h1 class="text-3xl font-bold text-gray-800">Gestor de Incidencias</h1>
8             <p class="text-gray-500 text-sm mt-1">
9               Mostrando página {{ tickets.current_page || 1 }} de {{ tickets.last_page || 1 }}
10              (Total: {{ tickets.total || 0 }} registros)
11            </p>
12          </div>
13          <button class="bg-green-600 hover:bg-green-700 text-white font-bold py-2 px-5 rounded-lg shadow transition flex items-center gap-2">
14            <svg xmlns="http://www.w3.org/2000/svg" class="h-5 w-5" fill="none" viewBox="0 0 24 24" stroke="currentColor">
15              <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M4 16v1a3 3 0 003 3h10a3 3 0 003-3v-1m-4-4l-4 4m0 0l-4-4m4 4V4" />
16            </svg>
17            Exportar Excel (CSV)
18          </button>
19        </div>
20
21        <TicketFilters @filter-applied="updateFiltersAndFetch" />
```

```
22
23        <div v-if="loading" class="flex justify-center items-center py-20 bg-white rounded-xl shadow border border-gray-200">
24          <div class="text-xl text-blue-600 font-semibold animate-pulse flex items-center gap-3">
25            Cargando datos...
26          </div>
27        </div>
28
29        <div v-else class="bg-white rounded-xl shadow-md overflow-hidden border border-gray-200">
30          <div class="overflow-x-auto">
31            <table class="min-w-full divide-y divide-gray-200">
32              <thead class="bg-gray-50">
33                <tr>
34                  <th scope="col" class="px-6 py-4 text-left text-xs font-bold text-gray-500 uppercase">ID</th>
35                  <th scope="col" class="px-6 py-4 text-left text-xs font-bold text-gray-500 uppercase">Título</th>
36                  <th scope="col" class="px-6 py-4 text-left text-xs font-bold text-gray-500 uppercase">Estado</th>
37                  <th scope="col" class="px-6 py-4 text-left text-xs font-bold text-gray-500 uppercase">Categoría</th>
38                  <th scope="col" class="px-6 py-4 text-left text-xs font-bold text-gray-500 uppercase">Fecha</th>
39                </tr>
40              </thead>
41              <tbody class="bg-white divide-y divide-gray-200">
42                <tr v-for="ticket in tickets.data" :key="ticket.id" class="hover:bg-blue-50 transition duration-150">
43                  <td class="px-6 py-4 whitespace-nowrap text-sm font-bold text-gray-900">#{{ ticket.id }}</td>
44                  <td class="px-6 py-4 whitespace-nowrap text-sm text-gray-700">{{ ticket.title }}</td>
45                  <td class="px-6 py-4 whitespace-nowrap text-sm">
46                    <span :class="{
47                      'bg-green-100 text-green-800': ticket.status === 'Resuelto',
48                      'bg-red-100 text-red-800': ticket.status === 'Abierto',
49                      'bg-yellow-100 text-yellow-800': ticket.status === 'Pendiente',
50                      'bg-gray-100 text-gray-800': ticket.status === 'Cerrado'
51                    }" class="px-3 py-1 inline-flex text-xs font-semibold rounded-full">
52                      {{ ticket.status }}
53                    </span>
54                  </td>
55                  <td class="px-6 py-4 whitespace-nowrap text-sm text-gray-600 font-medium">
56                    {{ ticket.category ? ticket.category.name : 'Sin categoría' }}
57                  </td>
58                  <td class="px-6 py-4 whitespace-nowrap text-sm text-gray-500">
59                    {{ new Date(ticket.created_at).toLocaleDateString('es-ES') }}
60                  </td>
61                </tr>
62              </tbody>
```

```
          </table>
        </div>

        <div class="bg-gray-50 px-6 py-4 border-t border-gray-200 flex items-center justify-between">
          <button @click="fetchTickets(tickets.current_page - 1)" :disabled="tickets.current_page === 1" class="bg-white border border-gray-300 text-gray-700 font-semibold py-2 px-4 rounded shadow-sm hover:bg-gray-50 disabled:opacity-50 transition">Anterior</button>
          <span class="text-sm text-gray-600">Página <span class="font-bold">{{ tickets.current_page }}</span> de {{ tickets.last_page }}</span>
          <button @click="fetchTickets(tickets.current_page + 1)" :disabled="tickets.current_page === tickets.last_page" class="bg-white border border-gray-300 text-gray-700 font-semibold py-2 px-4 rounded shadow-sm hover:bg-gray-50 disabled:opacity-50 transition">Siguiente</button>
        </div>
      </div>

    </div>
  </div>
</template>
```

```
<script setup>
import { ref, onMounted } from 'vue';
import TicketFilters from './components/TicketFilters.vue'; // Importo el componente

const tickets = ref({ data: [], current_page: 1, last_page: 1, total: 0 });
const loading = ref(true);
const activeFilters = ref({}); // Guardo los filtros que nos envíe el componente

// Esta función se dispara cuando el usuario pulsa "Aplicar Filtros" en el componente hijo
const updateFiltersAndFetch = (newFilters) => {
  activeFilters.value = newFilters;
  fetchTickets(1); // Siempre que filtro, volvemos a la página 1
};

const fetchTickets = async (page = 1) => {
  loading.value = true;
  try {
    const params = new URLSearchParams({ page: page });

    // Recorro los filtros activos y los añado a la URL si tienen algún valor
    Object.keys(activeFilters.value).forEach(key => {
      if (activeFilters.value[key] !== '') {
        params.append(key, activeFilters.value[key]);
      }
    });

    const response = await fetch(`/api/tickets?${params.toString()}`);
    const data = await response.json();
    tickets.value = data;
  } catch (error) {
    console.error("Error cargando los tickets:", error);
  } finally {
    loading.value = false;
  }
};

onMounted(() => {
  fetchTickets();
});
</script>
```

El componente de filtrado lo hago en un componente a parte para separar responsabilidades.

Para ello creo el componente de filtros. Creo la carpeta components dentro de resources/js y dentro creo el componente TicketFilters.vue

App.vue M | TicketFilters.vue U

resources > js > components > TicketFilters.vue > {} script setup

```vue
<template>
  <div class="bg-white p-5 rounded-xl shadow-sm border border-gray-200 mb-6">
    <div class="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-5 gap-4 items-end">

      <div class="lg:col-span-2">
        <label class="block text-xs font-bold text-gray-700 uppercase mb-1">Buscar Incidencia</label>
        <input
          v-model="filters.search"
          type="text"
          placeholder="Ej: Tubería, farola..."
          class="w-full bg-gray-50 border border-gray-300 text-gray-700 rounded-lg focus:ring-blue-500 focus:border-blue-500 px-4 py-2"
        >
      </div>

      <div>
        <label class="block text-xs font-bold text-gray-700 uppercase mb-1">Estado</label>
        <select v-model="filters.status" class="w-full bg-gray-50 border border-gray-300 text-gray-700 rounded-lg focus:ring-blue-500 focus:border-blue-500 px-4 py-2">
          <option value="">Todos</option>
          <option value="Abierto">Abierto</option>
          <option value="Pendiente">Pendiente</option>
          <option value="Resuelto">Resuelto</option>
          <option value="Cerrado">Cerrado</option>
        </select>
      </div>

      <div>
        <label class="block text-xs font-bold text-gray-700 uppercase mb-1">Desde Fecha</label>
        <input v-model="filters.date_from" type="date" class="w-full bg-gray-50 border border-gray-300 text-gray-700 rounded-lg focus:ring-blue-500 focus:border-blue-500 px-4 py-2">
      </div>

      <div>
        <label class="block text-xs font-bold text-gray-700 uppercase mb-1">Hasta Fecha</label>
        <input v-model="filters.date_to" type="date" class="w-full bg-gray-50 border border-gray-300 text-gray-700 rounded-lg focus:ring-blue-500 focus:border-blue-500 px-4 py-2">
      </div>

    </div>

    <div class="mt-5 flex justify-end gap-3 border-t border-gray-100 pt-4">
      <button @click="clearFilters" class="px-5 py-2 text-sm font-semibold text-gray-600 bg-gray-100 hover:bg-gray-200 rounded-lg transition">
        Limpiar Filtros
      </button>
      <button @click="applyFilters" class="px-5 py-2 text-sm font-bold text-white bg-blue-600 hover:bg-blue-700 rounded-lg shadow transition flex items-center gap-2">
        <svg xmlns="http://www.w3.org/2000/svg" class="h-4 w-4" fill="none" viewBox="0 0 24 24" stroke="currentColor">
          <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M3 4a1 1 0 011-1h16a1 1 0 011 1v2.586a1 1 0 01-.293.707l-6.414 6.414a1 1 0 00-.293.707V17l-4 4v-6.586a1 1 0 00-.293-.707L3.293 7.293A1 1 0 013 6.586V4z" />
        </svg>
        Aplicar Filtros
      </button>
    </div>
  </div>
</template>
```

```vue
<script setup>
import { ref } from 'vue';

// Definimos el evento que este componente emitirá hacia App.vue
const emit = defineEmits(['filter-applied']);

// Estado inicial vacío
const defaultFilters = {
  search: '',
  status: '',
  date_from: '',
  date_to: ''
};

// Variable reactiva que el usuario modifica con los inputs
const filters = ref({ ...defaultFilters });

// Envía los filtros al componente padre (App.vue)
const applyFilters = () => {
  emit('filter-applied', { ...filters.value });
};

// Resetea los campos y envía la orden de limpiar al padre
const clearFilters = () => {
  filters.value = { ...defaultFilters };
  emit('filter-applied', { ...filters.value });
};
</script>
```

Cierro la feature

git add .

git commit -m "feat: creado componente TicketFilters.vue y conectado a App.vue para filtrado dinámico"

git push

Corro los servidores y abro la app en el navegador

En una terminal ejecuto el servidor de Laravel

php artisan serve

```
@ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $
@ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $ php artisan serve
  INFO  Server running on [http://127.0.0.1:8000].

  Press Ctrl+C to stop the server
```

Y, en otro, el servidor de Vite

npm run dev

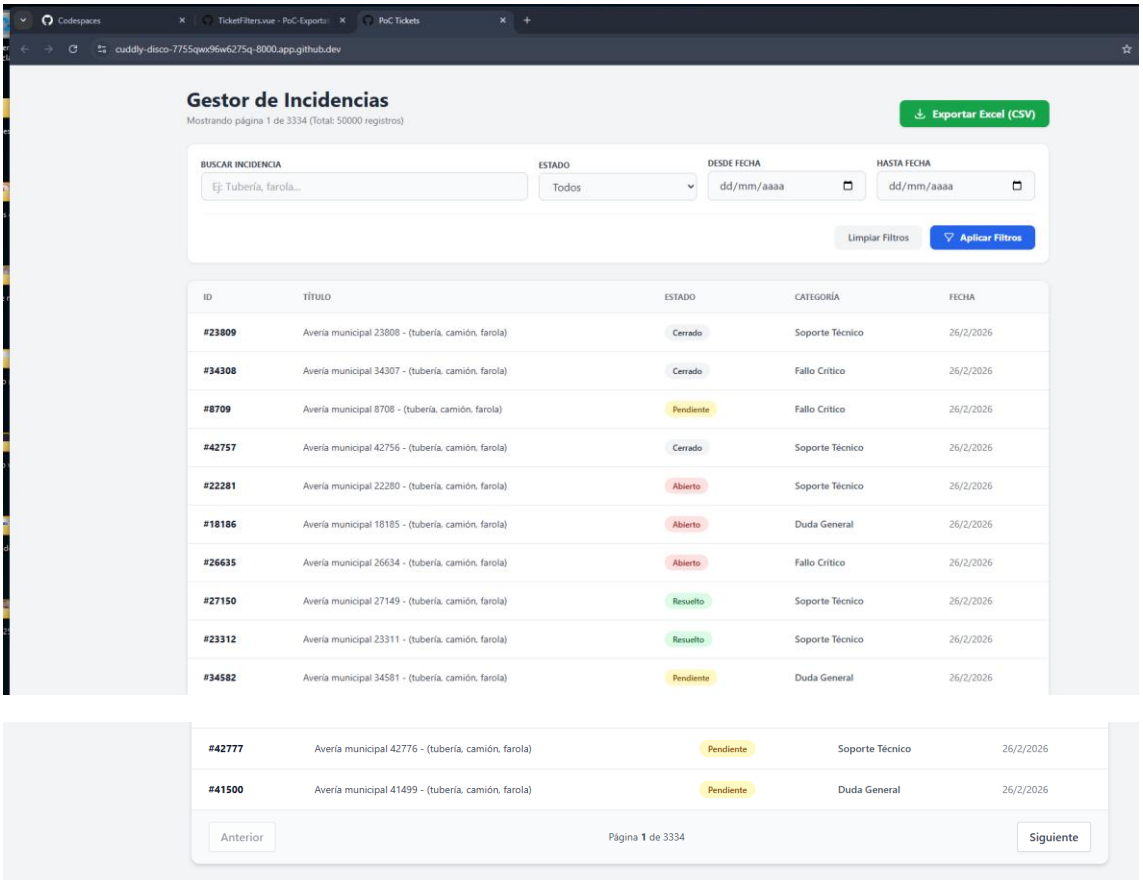
```
@ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $ npm run dev
  VITE v7.3.1  ready in 2348 ms

  ➜ Local:   http://localhost:5173/
  ➜ Network: http://10.0.3.168:5173/
  ➜ press h + enter to show help

  LARAVEL v12.51.0  plugin v2.1.0

  ➜ APP_URL: http://localhost
```

Hago los puertos públicos para acceder sin problemas:

Puerto	Dirección reenviada	Proceso en ejecución	Visibilidad	Origen
5173	https://cuddly-disco-7755qwx96w6275q-5173.app.github.dev/	node /workspaces/PoC-Exportation-Project/...	Public	Reenviado automáticamente
8000	https://cuddly-disco-7755qwx96w6275q-8000.app.github.dev/	/usr/bin/php8.3 -S 127.0.0.1:8000 /workspac...	Public	Reenviado por el usuario

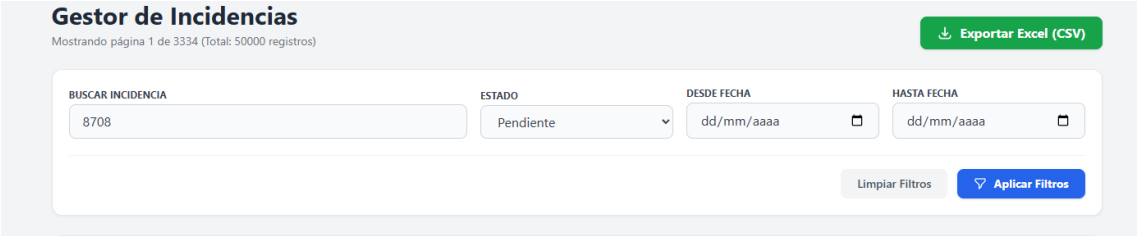
Agregar puerto

Abro la app en el navegador:



Pruebo la funcionalidad de los filtros

Para ello introduzco por ejemplo el texto 8708 y estado pendiente.



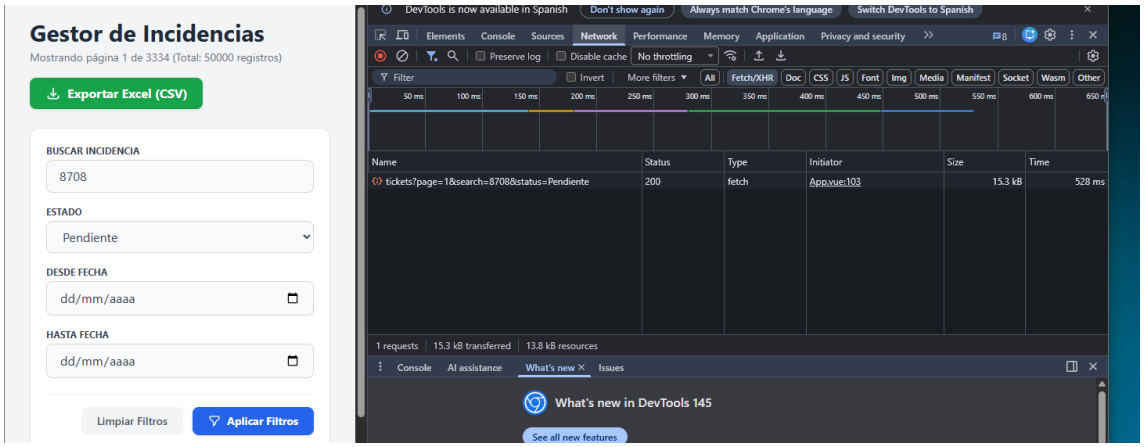
Aplico el filtro y compruebo que tengo un bug. Me devuelve todos los tickets de nuevo sin filtrar

Haciendo el diagnóstico aplico la técnica de seguimiento de la petición HTTP para encontrar en qué punto se rompe la comunicación:

Para comprobar que la petición se hace correctamente uso la pestaña Network de las DevTools filtrando solo las conexiones de datos Fetch/XHR cuando aplico el filtro:

Vuelvo a añadir la parte del título a filtrar: 8708

Vuelvo a añadir el estado: Pendiente



Veo que el fetch lo está haciendo correctamente y que me devuelve el estado 200 por lo que el fallo no está aquí.

El servicio está bien programado para recibir un array de filtros y aplicarlos a la base de datos.

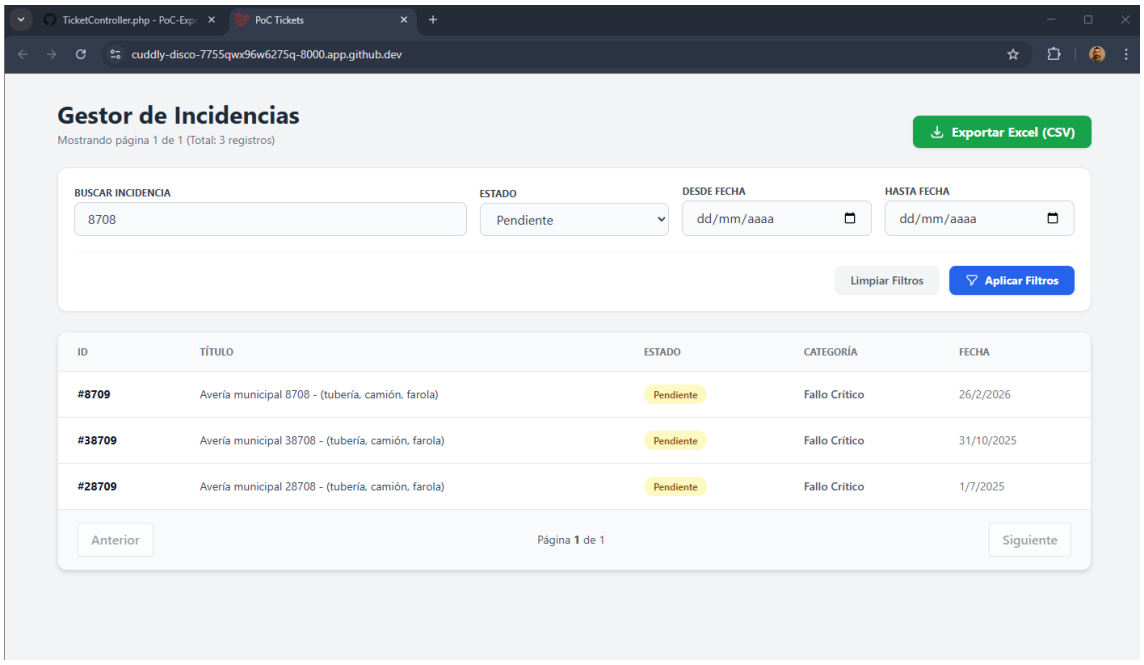
```php
/**
 * Obtiene los tickets filtrados y PAGINADOS (Para la tabla de Vue.js)
 */
public function getAllTickets(array $filters = [])
{
    // Usamos la consulta base y le aplicamos paginate(15)
    return $this->buildFilterQuery($filters)
                ->orderBy('created_at', 'desc')
                ->paginate(15);
}
```

Pero, encuentro el bug, veo que en el controlador la función index() olvidé pasar los parámetros por lo que el servicio entiende que no hay nada que filtrar y lo devuelve todo.

Reparo el controlador inyectando el objeto Request y pasándoselo como parámetro al servicio.

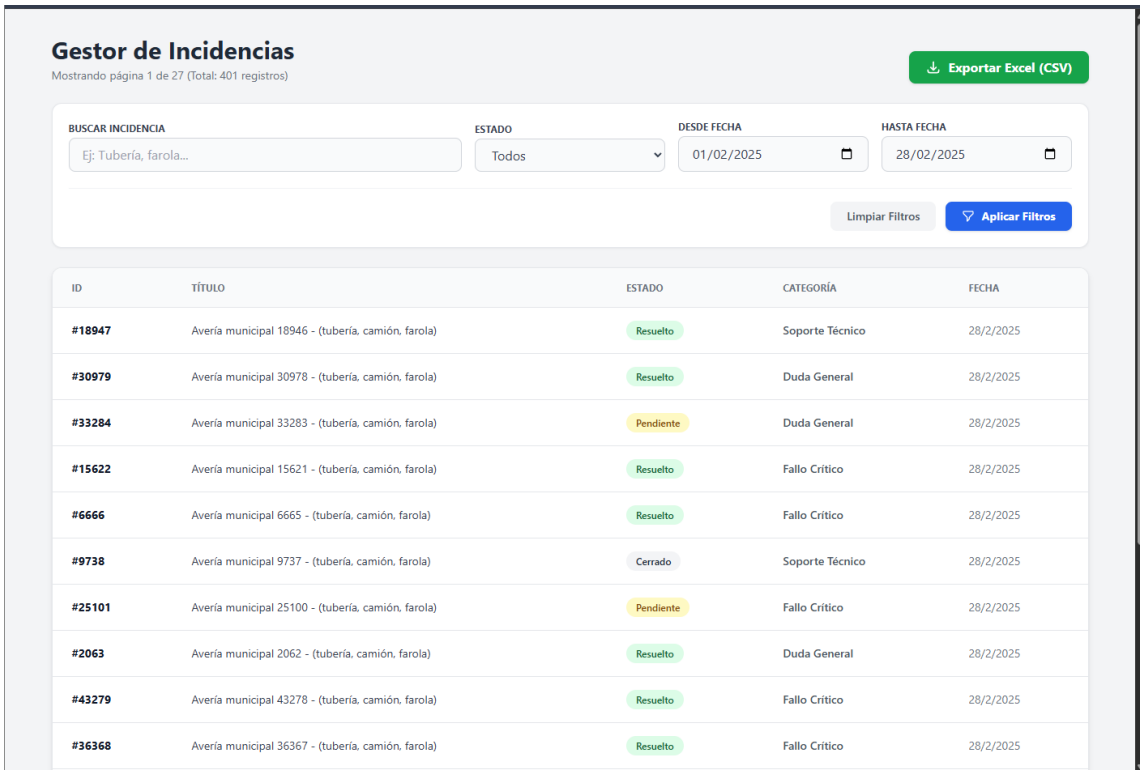
```php
public function index(Request $request)
{
    // El controlador ya no se encarga de buscar los tickets, solo se los pide al servicio
    $tickets = $this->ticketService->getAllTickets($request->all());

    return response()->json($tickets);
}
```

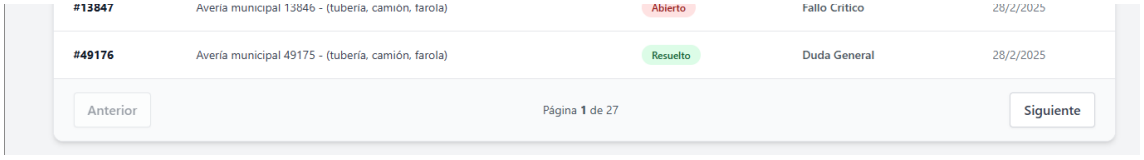
Vuelvo a correr los servidores en paralelo y pruebo de nuevo la funcionalidad de los filtros:



Ahora ya funciona.

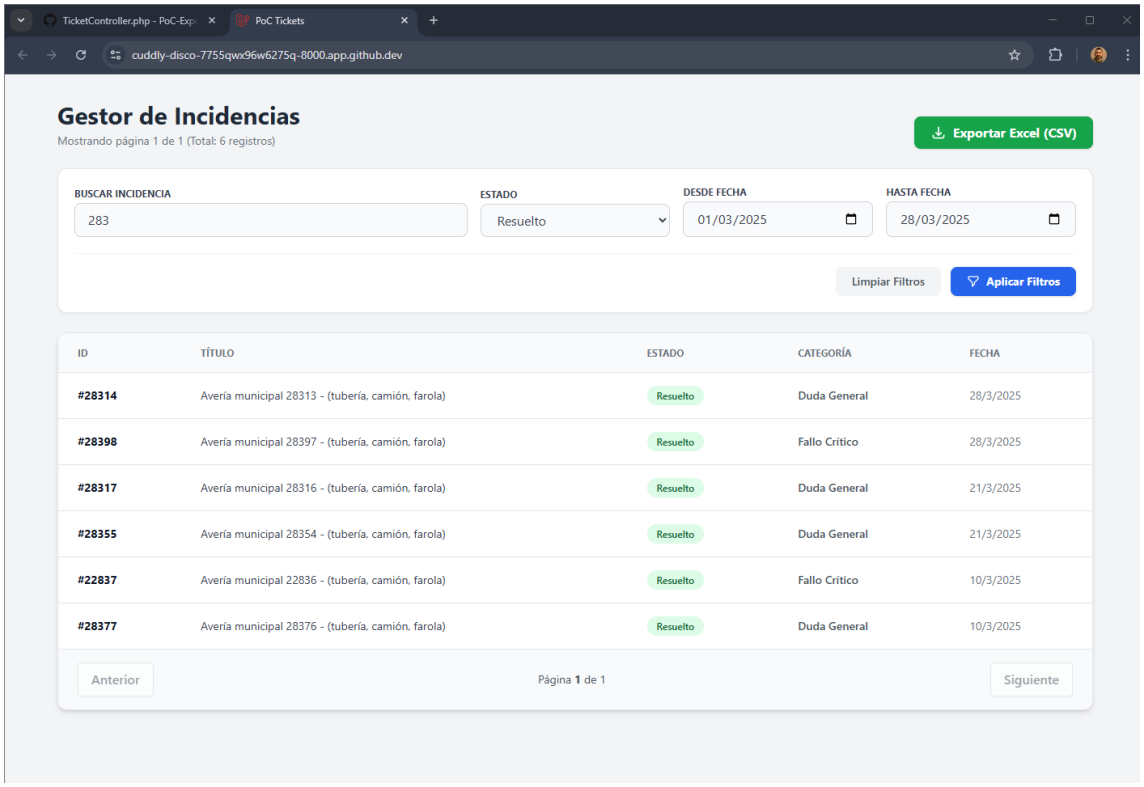
Pruebo el filtro de rango de fechas poniendo, por ejemplo, los del mes de febrero del 2025





Funciona el filtro.

Combino todos los filtros para probarlos



Funciona.

Hago el commit y actualizo el repositorio

git add .

git commit -m "aplicación con filtros funcionando"

git push

```
● @ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $ git add .
● @ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $ git commit -m "Aplicación funcinando con filtros"
  [develop b98780c] Aplicación funcinando con filtros
   4 files changed, 191 insertions(+), 8 deletions(-)
   create mode 100644 resources/js/components/TicketFilters.vue
● @ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $ git push
  Enumerating objects: 25, done.
  Counting objects: 100% (25/25), done.
  Delta compression using up to 2 threads
  Compressing objects: 100% (10/10), done.
  Writing objects: 100% (14/14), 4.09 KiB | 4.09 MiB/s, done.
  Total 14 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
  remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
  To https://github.com/ValentinuxX/PoC-Exportation-Project
     275711b..b98780c  develop -> develop
```

Comienzo a implementar la funcionalidad de la exportación masiva.

Para conectar el botón de Vue con el Backend la idea es, en lugar de cargar los 50.000 registros en la memoria de Laravel de golpe, lo iré leyendo de de la base de datos de uno en uno y los iré inyectando directamente en el archivo descargable del usuario.

En el controlador me doy cuenta de que en el método export en el foreach no estaba usando el cursor por lo que lo modifico:

```php
        // Iteramos sobre la colección y escribimos una línea por cada ticket
        foreach ($tickets->cursor() as $ticket) {
            fputcsv($file, [
                $ticket->id,
                $ticket->title,
                $ticket->status ?? 'Sin estado',
                // Uso el operador nullsafe (??) por si acaso algún ticket fue borrado de su usuario/categoría
                $ticket->user->name ?? 'Sin usuario',
                $ticket->category->name ?? 'Sin categoría',
                $ticket->category->priority ?? 'N/A', // Saco la prioridad desde la relación con la categoría
                $ticket->created_at->format('Y-m-d H:i:s')
            ]);
        }

        fclose($file);
    };
```

Me aseguro de que había expuesto la ruta correctamente:

```
App.vue    TicketController.php M    api.php    TicketFilters.vue
routes > api.php > ...
<?php

use Illuminate\Http\Request;
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\Api\TicketController; // importo el controlador

Route::get('/user', function (Request $request) {
    return $request->user();
})->middleware('auth:sanctum');

// La ruta de exportación
Route::get('/tickets/export', [TicketController::class, 'export']);

// Creo la ruta para la API de tickets (esta ruta crea GET, POST, PUT y DELETE automáticamente)
Route::apiResource('tickets', TicketController::class);
```

Añado la función para exportar en el componente App.vue:

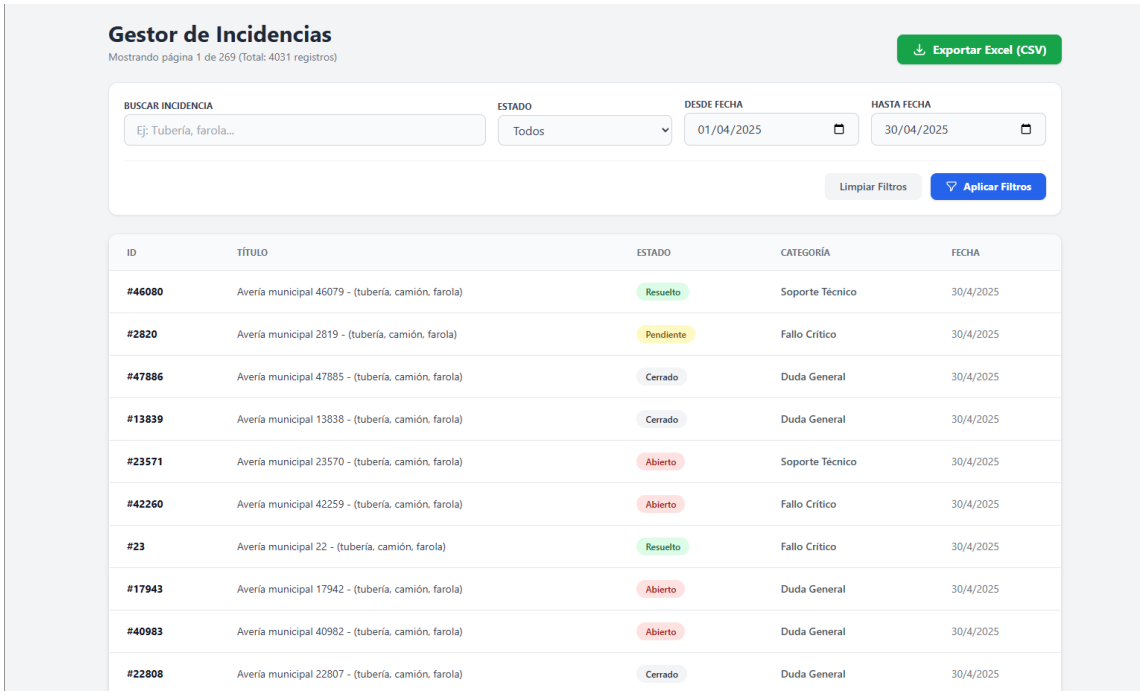
```vue
const exportCSV = () => {
  const params = new URLSearchParams();

  // Recorro los filtros activos (estado, fechas, búsqueda) y los añado a la URL
  Object.keys(activeFilters.value).forEach(key => {
    if (activeFilters.value[key] !== '') {
      params.append(key, activeFilters.value[key]);
    }
  });

  // Construyo la URL final hacia el método del controlador
  const exportUrl = `/api/tickets/export?${params.toString()}`;

  // Forzamos al navegador a "viajar" a esa URL para descargar el archivo.
  window.location.href = exportUrl;
};
</script>
```

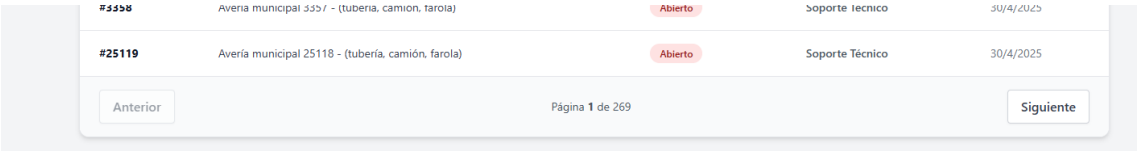
Y, ya en el template, la enlazo con el botón añadiendo el evento @click=”exportCSV”:

```
12        </div>
13        <button
14        @click="exportCSV"
15        class="bg-green-600 hover:bg-green-700 text-white font-bold py-2 px-5 rounded-lg shadow transition flex items-center gap-2"
16        >
17          <svg xmlns="http://www.w3.org/2000/svg" class="h-5 w-5" fill="none" viewBox="0 0 24 24" stroke="currentColor">
18            <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M4 16v1a3 3 0 003 3h10a3 3 0 003-3v-1m-4-4l-4 4m0 0l-4-4m4 4V4"
19          </svg>
20          Exportar Excel (CSV)
21        </button>
22      </div>
```

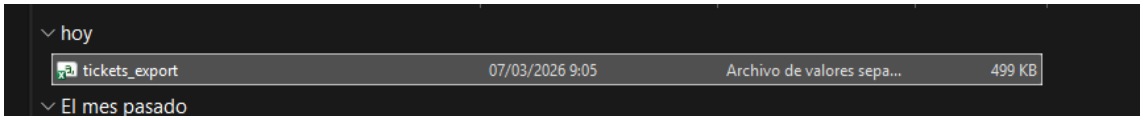
Hago la prueba final del funcionamiento de la exportación primero con filtro:

Hago un filtro buscando los tickets del mes de abril del 2025 (me salen 269 páginas):

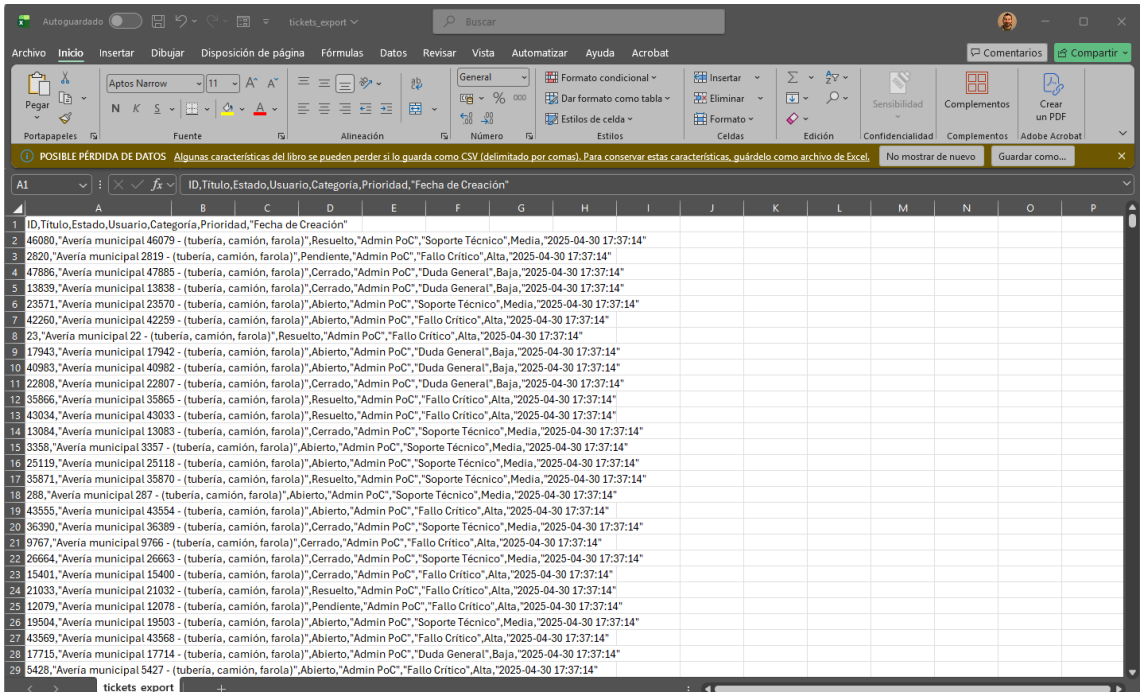


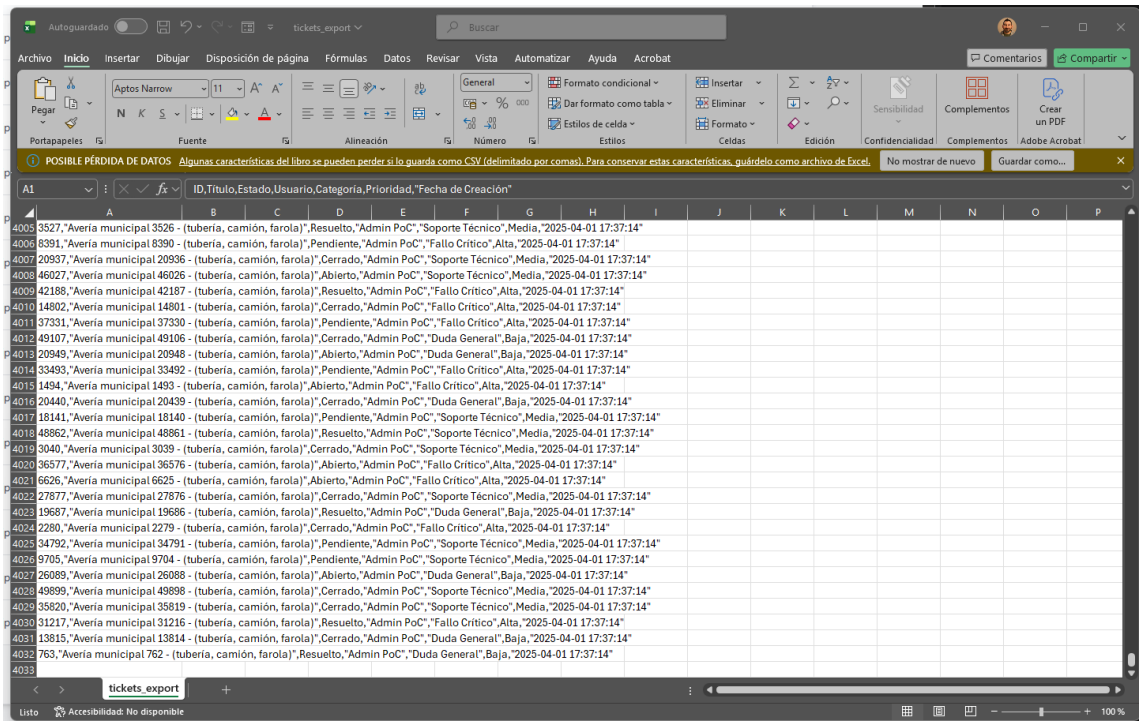


Pruebo a exportarlos:



Me exporta en apenas unos segundos 4032 filas de tickets:





Hago la exportación de los 50.000 tickets (debo optimizar)

Me ha tardado alrededor de 40 segundos la descarga, por lo que no es lo que estaba buscando, quiero optimizarlo más y la solución pasa por cambiar el foreach del método export y, en lugar de usar cursor() usar chunk() extrayendo 1000 tickets en cada iteración, con la que lograré recuperar la misma velocidad ya que sigo respetando el Eager Loading, pero manteniendo la memoria RAM plana y segura.

```
116
117             // Uso chunk() en lugar de cursor()
118             $tickets->chunk(1000, function ($chunk) use ($file) {
119                 foreach ($chunk as $ticket) {
120                     fputcsv($file, [
121                         $ticket->id,
122                         $ticket->title,
123                         $ticket->status ?? 'Sin estado',
124                         $ticket->user->name ?? 'Sin usuario',
125                         $ticket->category->name ?? 'Sin categoría',
126                         $ticket->category->priority ?? 'N/A',
127                         $ticket->created_at->format('Y-m-d H:i:s')
128                     ]);
129                 }
130             });
131             fclose($file);
132         };
133
```

Aho la prueba de la exportación masiva con los cambios:

Lo descarga en ~7 segundos.

El desafío crítico de esta PoC era la exportación masiva de 50.000 registros sin comprometer la estabilidad del servidor. Durante el desarrollo se auditaron tres enfoques arquitectónicos:

Enfoque en Memoria (->get()): Alta velocidad, pero riesgo crítico de desbordamiento de RAM. Inviable para producción.

Enfoque de Streaming Puro (->cursor()): Consumo nulo de RAM, pero penalización severa de tiempo (aprox. 40 segundos) debido al problema de consultas N+1 (Lazy Loading) al buscar relaciones complejas (Usuarios y Categorías).

Enfoque Híbrido Definitivo (->chunk() + Eager Loading + Streaming): Se alcanzó el equilibrio técnico. Al agrupar las peticiones en bloques de 1.000 registros y forzar la precarga de relaciones (with()), se redujeron las consultas a la base de datos de 100.000 a menos de 150. El resultado es un tiempo de exportación sostenido de ~7 segundos para 50.000 registros, manteniendo un consumo de RAM estable y seguro mediante response()->stream().

Próximos pasos en el proyecto general (usuarios y autenticación, roles, edición de tickets y pentesting).

Tengo en mente escalar esta prueba de concepto añadiendo funcionalidades como la autenticación, la gestión de usuarios y roles y la de poder crear tickets desde el frontend para, ya al final, hacer una “humilde” prueba de pentesting del proyecto y poner a prueba las posibles vulnerabilidades. Seguiré ese proyecto a partir de ahora, pero dando ya por concluida esta PoC.

Cierre de la PoC

Para cerrar la PoC voy a hacer el merge con la funcionalidad concluida en la rama main, para después volver a mi rama develop de nuevo y seguir añadiendo las funcionalidades de autenticación, gestión de usuarios y roles.

```
● @ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $ git add .
● @ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $ git commit -m "feat: optimización de exportación CSV con chunk y cierre de PoC"
  [develop f8f2464] feat: optimización de exportación CSV con chunk y cierre de PoC
   3 files changed, 36 insertions(+), 15 deletions(-)
● @ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $  git push origin develop
  Enumerating objects: 23, done.
  Counting objects: 100% (23/23), done.
  Delta compression using up to 2 threads
  Compressing objects: 100% (9/9), done.
  Writing objects: 100% (12/12), 1.52 KiB | 1.52 MiB/s, done.
  Total 12 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
  remote: Resolving deltas: 100% (4/4), completed with 4 local objects.
  To https://github.com/ValentinuxX/PoC-Exportation-Project
     6e048c7..f8f2464  develop -> develop
○ @ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $
```

```
● @ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $ git checkout main
  Switched to branch 'main'
  Your branch is up to date with 'origin/main'.
● @ValentinuxX ➜/workspaces/PoC-Exportation-Project (main) $ git merge develop
  Updating a9c91bc..f8f2464
  Fast-forward
   app/Http/Controllers/Api/TicketController.php |  32 ++++++------
   app/Services/TicketService.php                | 101 +++++++++++++++++----------------
   resources/js/App.vue                          | 136 +++++++++++++++++++++++++++++++++++++++++++++++++++
   resources/js/app.js                           |   5 ++
   resources/js/components/TicketFilters.vue     |  80 ++++++++++++++++++++++++++
   resources/views/welcome.blade.php             | 287 +++++------------------------------------------------------------------------------------------
   tests/Feature/TicketServiceTest.php           |  78 ++++++++++++++++++++++++++
   vite.config.js                                |  15 ++++++
   8 files changed, 393 insertions(+), 341 deletions(-)
   create mode 100644 resources/js/App.vue
   create mode 100644 resources/js/components/TicketFilters.vue
   create mode 100644 tests/Feature/TicketServiceTest.php
● @ValentinuxX ➜/workspaces/PoC-Exportation-Project (main) $ git push origin main
  Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
  To https://github.com/ValentinuxX/PoC-Exportation-Project
     a9c91bc..f8f2464  main -> main
● @ValentinuxX ➜/workspaces/PoC-Exportation-Project (main) $ git checkout develop
  Switched to branch 'develop'
  Your branch is up to date with 'origin/develop'.
○ @ValentinuxX ➜/workspaces/PoC-Exportation-Project (develop) $
```

Guardo el repositorio en mi equipo personal:

```
C:\Users\ka0_s>D:

D:\>cd "PoC Laravel y Vue"

D:\PoC Laravel y Vue>cd Repositorio

D:\PoC Laravel y Vue\Repositorio>cd PoC-Exportation-Project

D:\PoC Laravel y Vue\Repositorio\PoC-Exportation-Project>git fetch origin
remote: Enumerating objects: 23, done.
remote: Counting objects: 100% (23/23), done.
remote: Compressing objects: 100% (5/5), done.
remote: Total 12 (delta 4), reused 12 (delta 4), pack-reused 0 (from 0)
Unpacking objects: 100% (12/12), 1.50 KiB | 24.00 KiB/s, done.
From https://github.com/ValentinuxX/PoC-Exportation-Project
   6e048c7..f8f2464  develop    -> origin/develop
   a9c91bc..f8f2464  main       -> origin/main

D:\PoC Laravel y Vue\Repositorio\PoC-Exportation-Project>git checkout main
Switched to branch 'main'
Your branch is behind 'origin/main' by 4 commits, and can be fast-forwarded.
  (use "git pull" to update your local branch)

D:\PoC Laravel y Vue\Repositorio\PoC-Exportation-Project>git pull origin main
From https://github.com/ValentinuxX/PoC-Exportation-Project
 * branch            main       -> FETCH_HEAD
Updating a9c91bc..f8f2464
Fast-forward
 app/Http/Controllers/Api/TicketController.php |  32 +--
 app/Services/TicketService.php                | 101 ++++-----
 resources/js/App.vue                          | 136 ++++++++++++
 resources/js/app.js                           |   5 +
 resources/js/components/TicketFilters.vue     |  80 +++++++
 resources/views/welcome.blade.php             | 287 ++----------------------
 tests/Feature/TicketServiceTest.php           |  78 +++++++
 vite.config.js                                |  15 ++
 8 files changed, 393 insertions(+), 341 deletions(-)
 create mode 100644 resources/js/App.vue
 create mode 100644 resources/js/components/TicketFilters.vue
 create mode 100644 tests/Feature/TicketServiceTest.php
```

```
D:\PoC Laravel y Vue\Repositorio\PoC-Exportation-Project>git checkout develop
Switched to branch 'develop'
Your branch is behind 'origin/develop' by 1 commit, and can be fast-forwarded.
  (use "git pull" to update your local branch)

D:\PoC Laravel y Vue\Repositorio\PoC-Exportation-Project>git pull origin develop
From https://github.com/ValentinuxX/PoC-Exportation-Project
 * branch            develop    -> FETCH_HEAD
Updating 6e048c7..f8f2464
Fast-forward
 app/Http/Controllers/Api/TicketController.php | 28 +++++++++++++--------------
 resources/js/App.vue                          | 22 ++++++++++++++++++++-
 resources/js/components/TicketFilters.vue     |  1 +
 3 files changed, 36 insertions(+), 15 deletions(-)

D:\PoC Laravel y Vue\Repositorio\PoC-Exportation-Project>
```

Repositorio público:

Repositorio: https://github.com/ValentinuxX/PoC-Exportation-Project